# *k*NN Algorithm with Data-Driven *k* Value

Debo Cheng, Shichao Zhang*, Zhenyun Deng, Yonghua Zhu**, and Ming Zong

College of Computer Science & Information Technology,
Guangxi Normal University, Guilin, Guangxi, 541004, China
zhangsc@mailbox.gxnu.edu.cn

**Abstract.** This paper proposes a new $k$ Nearest Neighbor ($k$NN) algorithm based on sparse learning, so as to overcome the drawbacks of the previous $k$NN algorithm, such as the fixed $k$ value for each test sample and the neglect of the correlation of samples. Specifically, the paper reconstructs test samples by training samples to learn the optimal $k$ value for each test sample, and then uses $k$NN algorithm with the learnt $k$ value to conduct all kinds of tasks, such as classification, regression, and missing value imputation. The rationale of the proposed method is that different test samples should be assigned different $k$ values in $k$NN algorithm, and learning the optimal $k$ value for each test sample should be taken the correlation of data into account. To this end, in the reconstruction process, the proposed method is designed to achieve the minimal reconstruction error via a least square loss function, and employ an $\ell_1$-norm regularization term to create the element-wise sparsity in the reconstruction coefficient, *i.e.,* sparsity appearing in the element of the coefficient matrix. For achieving effectiveness, the Locality Preserving Projection (LPP) is employed to keep the local structures of data. Finally, the experimental results on real datasets, and the experimental results show that the proposed $k$NN algorithm is better than the state-of-the-art algorithms in terms of different learning tasks, such as classification, regression, and missing value imputation.

**Keywords:** Sparse learning, k nearest neighbors, locality preserving projection, classification.

## 1 Introduction

The $k$ Nearest Neighbors algorithm ($k$NN for short) is an instance-based, or a lazy learning method. It has been regarded as one of the simplest of all machine learning algorithms [4][13]. The rational of $k$NN is that similar samples belonging to the same class have high probability, while the key idea of $k$NN algorithm is to first select $k$ nearest neighbors for each test sample, followed by using the learnt $k$ nearest neighbors to predict this test sample. Therefore, $k$NN algorithm was often thought as an algorithm, in which no explicit training step is required.

* Corresponding author.

** Yonghua Zhu participated this work when he was a visiting undergraduate student of the Guangxi Key Lab of Multi-source Information Mining & Security (MIMS) of Guangxi Normal University, supervised by Professor Shichao Zhang. He is currently with School of Computer, Electronics and Information, Guangxi University, Nanning, Guangxi, 530004, China.

X. Luo, J.X. Yu, and Z. Li (Eds.): ADMA 2014, LNAI 8933, pp. 499–512, 2014.
© Springer International Publishing Switzerland 2014

A drawback of the $k$NN algorithm is that it is sensitive to select the value of $k$. Although studies have been focusing on this topic for a long time, the selection of $k$ value in $k$NN algorithm is still very difficult and challengeable [8][11]. For example, Lall and Sharama mentioned that the suitable setting of $k$ value should satisfy $k = \sqrt{n}$ for the datasets with a sample size larger than 100 [9]. However, such a setting has been proved to not suitable for all cases of datasets [10].

In this paper, we focus on the setting of $k$ value in $k$NN algorithm based on the following observations, in which there is a binary classification task (where two classes are marked as the symbol '+' and the symbol '-', respectively) and the test sample with no label is marked with the symbol '?' in Fig. 1.

In Fig. 1, by setting $k = 5$ in the $k$NN algorithm, both the two test samples will be assigned '+' class. Actually, according to the distribution of data in Fig. 1, it is reasonable for the class label of the first test sample. However, we can find that the second test sample should be assigned as '-' class (*i.e.,* $k = 3$) rather than '+' class (*i.e.,* $k = 5$). Moreover, we can also find the similar scenario in $k$NN regression or missing value imputation with $k$NN algorithm. According to our observations, the $k$ value in $k$NN algorithm should be data-driven, *i.e.,* decided by the distribution of data. Moreover, the $k$ value for each test sample might be different.

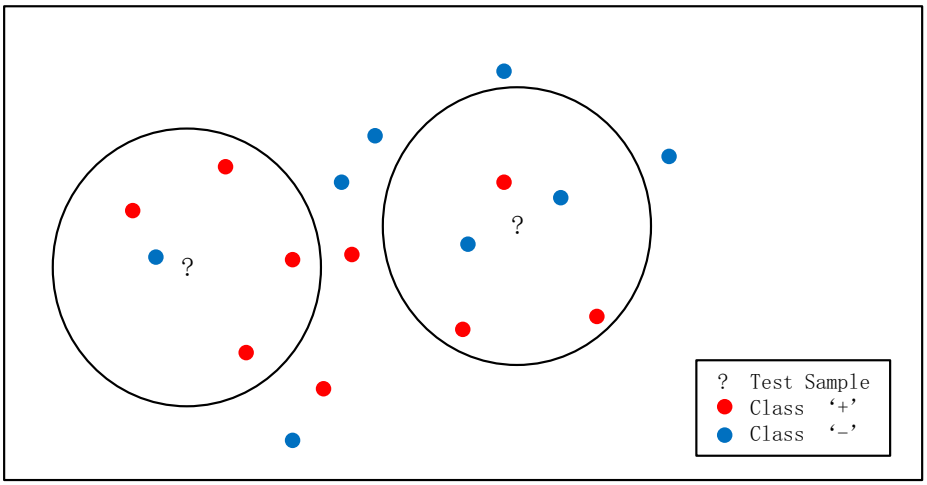



**Fig. 1.** An example of $k$NN classification task with $k = 5$

To overcome the above drawback of $k$NN algorithm, this paper proposes a Sparse learning based $k$NN method (S-$k$NN for short) to learn the optimal $k$ values for different test samples when using $k$NN algorithm. This is carried out by the consideration of both the correlation and the local structure of training samples [37]. Specifically, we first reconstruct test samples with training samples to obtain an optimal $k$ value for each test sample, and then use $k$NN algorithm with the learnt $k$ to conduct classification or regression or missing value imputation[18][16]. The key step of the proposed method is the reconstruction process, in which, we employ a least square loss function to achieve

the minimal reconstruction error, and use an $\ell_1$-norm regularization term to result in the element-wise sparsity for generating various $k$ values of different test samples. We also employ Locality Preserving Projection (LPP) [7] for preserving the local structures of data during the reconstruction process, aiming to further improve the reconstruction performance.

Comparing with the previous $k$NN algorithms, the proposed S-$k$NN algorithm has the following advantages:

- Different from the previous $k$NN algorithms that use either a fixed $k$ value for all test samples or the cross-validation method to select the $k$ value without considering the correlation of samples, the proposed S-$k$NN algorithm learns an optimal $k$ value for each test sample according to the distribution of data. Moreover, during the learning process, the proposed S-$k$NN algorithm considers the correlation among samples, for further improving the performance. Therefore, the proposed reconstruction process for learning the optimal $k$ values is data-driven.
- This paper proposes a new sparse learning framework for learning $k$ values. Different from conventional Least Absolute Shrinkage and Selection Operator (LASSO) [14][31], our sparse learning framework also takes the local structures of samples into account. Moreover, this paper proposes a novel optimization method to solve the designed objective function.
- Different from the conventional $k$NN algorithms using for a single task, this paper uses the designed S-$k$NN method for three tasks, such as classification, regression, and missing value imputation. Moreover, the experimental results on real datasets show the benefit of the proposed method by comparing to the state-of-the-art $k$NN algorithms.

The remainder of the paper is arranged as follows. We review the applications of $k$NN algorithm in Section 2 and then give the detail of the proposed method in Section 3. Furthermore, we analyze the experimental results in Section 4 and give our conclusion and future work in Section 5.

## 2 Related Work

The study of $k$NN method has been becoming a hot research topic in data mining and machine learning since the algorithm was proposed in 1967 [2][6]. In this section, we briefly review the applications of $k$NN algorithm in data mining tasks, such as classification, regression, and missing value imputation.

### 2.1 Classification

The $k$NN classification was designed to select the $k$ closest training samples to the test samples (*i.e.,* $k$ nearest neighbors) from all the known samples, followed by a simple classifier, *e.g.,* majority classification rule [20][24][17]. Liu proposed an anomaly removal algorithm under the framework of $k$NN algorithm, for conducting $k$NN classification [10]. Weinberger *et al.* first demonstrated the reasonability to learn a Mahanalobis distance metric for $k$NN classification, and then optimized the measure metric

with the goal that $k$NN always belong to the same class while examples form different classes are separated by a large margin [15]. Goldberger *et al.* proposed a neighborhood component analysis (NCA) method, *i.e.,* a new distance metric learning algorithm, to improved $k$NN classification [6]. Zhang incorporated Certainty Factor (CF) measure to deal with the unsuitability of skewed class distribution in the $k$NN framework [22].

### 2.2 Regression

The $k$NN algorithm for regression has been currently used and studied for many years in pattern recognition and data mining. In regression analysis, Burba *et al.* proposed some asymptotic properties of the $k$NN based kernel estimator for improving the performance of $k$NN regression [2]. Moreover, the goal of their paper is to study the nonparametric $k$NN algorithm. Ferraty and Vieu used the nonparametric characteristics of the $k$NN algorithm for conducting estimation, classification and discrimination on the high-dimensional data [5].

### 2.3 Missing Value Imputation

Missing data are often found in the study of data mining and machine learning. The learning task with missing data easily leads to low performance [12][25][38]. Although there are many solutions to deal with missing data, such as missing instance deletion and missing value imputation, and so on, the $k$NN based algorithms for missing value imputation are the key solutions in real applications [21][23][26][28]. For example, Zhang *et al.* proposed a Grey-Based $k$NN Iteration Imputation (GBKII) method, replacing Euclidean distance in conventional $k$NN algorithm with the grey-based distance measure, to improve the performance of $k$NN algorithm [19]. Recently, Zhang designed to select the left and right nearest neighbors of missing data for improving the performance of the previous $k$NN algorithm [27].

## 3 Method

In this section, we first introduce some basic concepts used in this paper. And then the S-$k$NN method is described. Finally, we give our optimization method to solve the proposed objective function.

### 3.1 Notation

Throughout the paper, we denote matrices as boldface uppercase letters, vectors as boldface lowercase letters, and scalars as normal italic letters, respectively. For a matrix $\mathbf{X} = [x_{ij}]$, its *i*-th row and *j*-th column are denoted as $\mathbf{x}^i$ and $\mathbf{x}_j$, respectively. Also, we denote the Frobenius norm, the $\ell_2$-norm, and the $\ell_1$-norm, respectively, as $\|\mathbf{X}\|_F = \sqrt{\sum_i \|\mathbf{x}^i\|_2^2} = \sqrt{\sum_j \|\mathbf{x}_j\|_2^2}$, $\|\mathbf{x}_j\|_2 = \sqrt{\sum_i^n x_{i,j}^2}$, and $\|\mathbf{X}\|_1 = \sum_i \sum_j |x_{i,j}|$, respectively. We further denote the transpose operator, the trace operator, and the inverse of a matrix $\mathbf{X}$ as $\mathbf{X}^T$, $tr(\mathbf{X})$, and $\mathbf{X}^{-1}$, respectively.

### 3.2 Reconstruction and Locality Preserving Projection (LPP)

Given $\mathbf{X} = \{\mathbf{x}_i\}_{i=1}^{n} \in \mathbb{R}^{n \times d}$ and $\mathbf{Y} = \{\mathbf{y}_i\}_{i=1}^{m} \in \mathbb{R}^{m \times d}$, where $n$, $m$, and $d$, represents the number of training samples, test samples, and feature dimension, respectively. To reconstruct test samples with training samples for obtaining the reconstruction coefficient matrix $\mathbf{W} \in \mathbb{R}^{n \times m}$, we define the following objective function [32][33][34]:

$$\arg\min_{\mathbf{W}} \sum\nolimits_i \left\| \mathbf{w}_i{}^T \mathbf{x}_i - \mathbf{y}_i \right\| \tag{1}$$

where $w_{i,j}$ is used to test the correlation between $\mathbf{y}_i$ and training sample $\mathbf{x}_j$. The larger the value of $w_{i,j}$, the larger correlation between the *i*-th test sample and the *j*-th training sample. In particular, the case of $w_{i,j} = 0$ indicates that there is no correlation between $\mathbf{y}_i$ and $\mathbf{x}_j$.

LPP is a nonlinear subspace learning method [7]. The rationale of LPP is to obtain an optimal linear transformation $\mathbf{W}$ so that the local structures of the original data are preserved in the new space, *i.e.,* $\mathbf{W}$ converts the high-dimensional data $\mathbf{X}$ into the low-dimensional data $\mathbf{Y}$ with the following definition:

$$\mathbf{y}_j = \mathbf{W}^T \mathbf{x}_i, i = 1, 2, \cdots, n \tag{2}$$

To this end, the objective function of LPP can be defined as follows:

$$\min_{\mathbf{W}} \sum_{i,j} \left( \mathbf{W}^T \mathbf{x}_i - \mathbf{W}^T \mathbf{x}_j \right)^2 s_{ij} \tag{3}$$

where $\mathbf{S}$ is the weight matrix and each element of $\mathbf{S}$ is defined by a heat kernel $s_{i,j} = exp(\frac{-\|\mathbf{x}_i - \mathbf{x}_j\|^2}{\sigma})$[1].

By plugging Eq. (2) into Eq. (3), and some algebraic transformation operations, we obtain:

$$\begin{aligned} & \frac{1}{2} \sum_{ij} \left( \mathbf{W}^T \mathbf{x}_i - \mathbf{W}^T \mathbf{x}_j \right)^2 s_{ij} \\ &= \sum_i \left( \mathbf{W}^T \mathbf{x}_i d_{ij} \mathbf{x}_i{}^T \mathbf{W} \right) - \sum_{ij} \left( \mathbf{W}^T \mathbf{x}_i s_{ij} \mathbf{x}_i{}^T \mathbf{W} \right) \\ &= tr(\mathbf{W}^T \mathbf{X} \mathbf{D} \mathbf{X}^T \mathbf{W}) - tr(\mathbf{W}^T \mathbf{X} \mathbf{S} \mathbf{X}^T \mathbf{W}) \\ &= tr(\mathbf{W}^T \mathbf{X} \mathbf{L} \mathbf{X}^T \mathbf{W}) \end{aligned} \tag{4}$$

where $\mathbf{D}$ is a diagonal matrix and the *i*-th diagonal element of $\mathbf{D}$ is defined as $d_{i,i} = \Sigma_j s_{i,j}$. Hence, $\mathbf{L} = \mathbf{D} - \mathbf{S}$ is a Laplacian matrix.

### 3.3 Approach

We reconstruct test samples $\mathbf{Y}$ with training samples $\mathbf{X}$ to obtain the linear transformation matrix $\mathbf{W}$. We expect to map $\mathbf{X}$ into the space of $\mathbf{Y}$ via $\mathbf{W}$ and make the distance

[1] $\sigma$ is a tuning parameter. For simplicity, we set $\sigma = 1$ in our experiments.

between $\mathbf{Y}$ and $\mathbf{W}^T\mathbf{X}$ as small as possible. To this end, we employ the least square loss function [35,36]:

$$\begin{aligned}\left\|\mathbf{W}^T\mathbf{X}-\mathbf{Y}\right\|_F^2 &= \left\|\hat{\mathbf{Y}}-\mathbf{Y}\right\|_F^2 \\ &= \sum_{i=1}^{n}\sum_{j=1}^{m}\left(y_{ij}-\hat{y}_{ij}\right)^2\end{aligned} \tag{5}$$

where $\hat{\mathbf{Y}}$ is the new representation of $\mathbf{X}$ in the space of $\mathbf{Y}$, *i.e.,* $\hat{\mathbf{Y}}=\mathbf{W}^T\mathbf{X}$. $\|\mathbf{W}^T\mathbf{X}-\mathbf{Y}\|_F^2$ represents the reconstruction error. Due to that Eq. (5) is convex, we can easily obtain its global solution $\mathbf{W}=(\mathbf{X}\mathbf{X}^T)^{-1}+\mathbf{X}\mathbf{Y}$. However, $\mathbf{X}\mathbf{X}^T$ is not always invertible, so an $\ell_2$-norm is often added to remove the issue of invertible. This leads to the ridge regression:

$$\arg\min_{\mathbf{W}}\left\|\mathbf{W}^T\mathbf{X}-\mathbf{Y}\right\|_F^2+\delta\left\|\mathbf{W}\right\|_2^2 \tag{6}$$

where $\delta$ is a tuning parameter. The optimal solution of Eq. (6) can be described as a closed solution $\mathbf{W}=(\mathbf{X}\mathbf{X}^T+\delta\mathbf{I})^{-1}+\mathbf{X}\mathbf{Y}$, where $\mathbf{I}\in\mathbb{R}^{n\times n}$ is an identity matrix.

The regularization term $\ell_1$-norm has been proved to generate sparsity, *i.e.,* zero in the elements of a matrix [30], while studies have shown that the $\ell_2$-norm did not surely generate sparse result. In this paper, the element $w_{i,j}$ indicates the correlation between the *i*-th test sample and the *j*-th training sample. We expect that each test sample is only represented by part of training samples, *i.e.,* many zero elements on each column in $\mathbf{W}$. Therefore, it makes sense for us to use an $\ell_1$-norm to replace the $\ell_2$-norm. Meanwhile, we also employ the LPP to preserve the local structures of data after the reconstruction process. To this end, the objective function of the proposed S-$k$NN method is defined as follows:

$$\arg\min_{\mathbf{W}}\frac{1}{2}\left\|\mathbf{Y}-\mathbf{W}^T\mathbf{X}\right\|_F^2+\rho_1 tr(\mathbf{W}^T\mathbf{X}\mathbf{L}\mathbf{X}^T\mathbf{W})+\rho_2\|\mathbf{W}\|_1 \tag{7}$$

where both $\rho_1$ and $\rho_2$ are tuning parameters. $\rho_1$ is designed to balance the magnitude between $tr(\mathbf{W}^T\mathbf{X}\mathbf{L}\mathbf{X}^T\mathbf{W})$ and $\left\|\mathbf{Y}-\mathbf{W}^T\mathbf{X}\right\|_F^2$. Moreover, the larger the value of $\rho_1$, the larger the contribution of LPP in Eq. (7). In particular, Eq. (7) shrinks to LASSO while setting $\rho_1=0$.

Different from LASSO, the proposed S-$k$NN considers preserving the local structures of data via LPP. Moreover, our S-$k$NN is used to learn the *k* value of $k$NN algorithm. Different from that the conventional $k$NN algorithms assign the fixed *k* value for all test samples or learn the *k* value for each test sample without considering the correlation among test samples, the proposed S-$k$NN algorithm learns the optimal *k* value for each test sample via a reconstruction process. During the reconstruction process, the proposed method considers the correlation of test samples and training samples. More specifically, the proposed method considers the correlation of test samples through generating the *k* values for all test samples once, and considers the correlation of training samples through adding the LPP regularization term in the reconstruction process. Moreover, the proposed S-$k$NN method is a data-driven method for selecting the optimal *k* values.

### 3.4 Optimization

Eq. (7) is a convex but non-smooth function. In this work, we solve it by designing a new accelerated proximal gradient method [37]. We first conduct the proximal gradient method on Eq. (7) by letting:

$$f(\mathbf{W}) = \frac{1}{2}\left\|\mathbf{Y} - \mathbf{W}^T\mathbf{X}\right\|_F^2 + \rho_1 tr(\mathbf{W}^T\mathbf{X}\mathbf{L}\mathbf{X}^T\mathbf{W}) \tag{8}$$

$$\vartheta(\mathbf{W}) = f(\mathbf{W}) + \rho_2\|\mathbf{W}\|_1 \tag{9}$$

Note that $f(\mathbf{W})$ is convex and differentiable. To optimize $\mathbf{W}$ with the proximal gradient method, we iteratively update it by means of the following optimization rule:

$$\mathbf{W}(t+1) = \arg\min_{\mathbf{W}} \mathbf{G}_{\eta(t)}(\mathbf{W}, \mathbf{W}(t)) \tag{10}$$

where $\mathbf{G}_{\eta(t)}(\mathbf{W}, \mathbf{W}(t)) = f(\mathbf{W}(t)) + \langle \nabla f(\mathbf{W}(t)), \mathbf{W} - \mathbf{W}(t)\rangle + \frac{\eta(t)}{2}\|\mathbf{W} - \mathbf{W}(t)\|_F^2 + \rho_2\|\mathbf{W}\|_1$, $\nabla f(\mathbf{W}(t)) = (\mathbf{X}\mathbf{X}^T + \rho_1\mathbf{X}\mathbf{L}\mathbf{X}^T)\mathbf{W}(t) - \mathbf{X}\hat{\mathbf{Y}}^T$, $\langle\cdot,\cdot\rangle$ is an inner product operator, $\eta(t)$ determines the stepsize of the *t*- iteration, and $\mathbf{W}(t)$ is the value of $\mathbf{W}$ obtained at the $t$-iteration.

By ignoring the terms independent of $\mathbf{W}$ in Eq. (10), we can rewrite it as follows:

$$\mathbf{W}(t+1) = \pi_{\eta(t)}(\mathbf{W}(t)) = \arg\min \frac{1}{2}\|\mathbf{W} - \mathbf{U}(t)\|_2^2 + \frac{\rho_2}{\eta(t)}\|\mathbf{W}\|_1 \tag{11}$$

where $\mathbf{U}(t) = \mathbf{W}(t) - \frac{1}{\eta(t)}\nabla f(\mathbf{W}(t))$ and $\pi_{\eta(t)}(\mathbf{W}(t))$ is the Euclidean projection of $\mathbf{W}(t)$ onto the convex set $\eta(t)$. Due to the separability of $\mathbf{W}(t+1)$ on each row, *i.e.,* $\mathbf{w}^i(t+1)$, we update the weights for each row individually:

$$\mathbf{w}^i(t+1) = \arg\min_{\mathbf{w}^i} \frac{1}{2}\left\|\mathbf{w}^i - \mathbf{u}^i(t)\right\|_2^2 + \frac{\rho_2}{\eta(t)}\left\|\mathbf{w}^i\right\|_2 \tag{12}$$

where $\mathbf{u}^i(t) = \mathbf{w}^i(t) - \frac{1}{\eta(t)}\nabla f(\mathbf{w}^i(t))$. In Eq. (12), $\mathbf{w}^i(t+1)$ takes a closed form solution [36] as follows:

$$\mathbf{w}^{i*} = max\{\left|\mathbf{w}^i\right| - \rho_2, 0\} \cdot sgn(\mathbf{w}^i) \tag{13}$$

Meanwhile, in order to accelerate the proximal gradient method in Eq. (8), we further introduce an auxiliary variable $\mathbf{V}(t+1)$ as follows:

$$\mathbf{V}(t+1) = \mathbf{W}(t) + \frac{\alpha(t) - 1}{\alpha(t+1)}(\mathbf{W}(t+1) - \mathbf{W}(t)) \tag{14}$$

where the coefficient $\alpha(t+1)$ is usually set as $\alpha(t+1) = \frac{1+\sqrt{1+4\alpha(t)^2}}{2}$.

Finally, we list the pseudo of our proposed optimization method in Algorithm 1 and its convergence in Theorem 1.

**Algorithm 1.** Pseudo code of solving Eq. (7).

**Input**: $\eta(0) = 0.01, \alpha(1) = 1, \gamma = 0.002, \rho_1, \rho_2$;
**Output**: $\mathbf{W}$;
Initialize $t = 1$;
Initialize $\mathbf{W}(1)$ as a random diagonal matrix;
**repeat**
  **while** $L(\mathbf{W}(t)) > G_{\eta(t-1)}(\pi_{\eta(t-1)}(\mathbf{W}(t)), \mathbf{W}(t))$ **do**
    Set $\eta(t-1) = \gamma\eta(t-1)$;
  **end**
  Set $\eta(t) = \eta(t-1)$;
  Compute $\mathbf{W}(t+1) = \arg\min_{\mathbf{W}} G_{\eta(t)}(\mathbf{W}, \mathbf{V}(t))$;
  Compute $\alpha(t+1) = \frac{1+\sqrt{1+4\alpha(t)^2}}{2}$;
  Compute Eq. (14);
**until** *Eq. (7) converges*;

**Theorem 1.** *[36] Assume* $\{\mathbf{W}(t)\}$ *be the sequence obtained by Algorithm 1, then for* $\forall\, t \geq 1$*, the following holds*

$$\vartheta(\mathbf{W}(t)) - \vartheta(\mathbf{W}^*) \leq \frac{2\gamma L \left\|\mathbf{W}(1) - \mathbf{W}^*\right\|_{\mathrm{F}}^2}{(t+1)^2} \tag{15}$$

*where* $\gamma > 0$ *is a predefined constant, L is the Lipschitz constant of the gradient of* $f(\mathbf{W})$ *in Eq. (8), and* $\mathbf{W}^* = \arg\min_{\mathbf{W}} \vartheta(\mathbf{W})$.

Theorem 1 shows that the convergence rate of the proposed accelerated proximal gradient method in Algorithm 1 is $\mathcal{O}(\frac{1}{t^2})$, where *t* denotes an iteration number.

### 3.5 Algorithm

In this paper, we first optimize Eq. (7) to obtain the correlation coefficient matrix $\mathbf{W}$, aiming to obtain an optimal *k* value for each test sample. We then use the selected *k* to conduct $k$NN algorithm for different tasks, such as classification, regression, and missing value imputation. The pseudo of S-$k$NN is presented in Algorithm 2.

In both the regression task and missing value imputation, the bigger the correlation between a test sample and its nearest neighbors[29], the larger the contribution of the nearest neighbors to the test sample. Therefore, this paper proposes a weighted method for the prediction of both the regression task and missing value imputation task. Specifically, the weighted predictive value of the *j*-th test sample is defined as:

$$\mathrm{predictValue_weight} = \sum_{i=1}^{n} \left(\frac{w_{i,j}}{\sum_{i=1}^{n} w_{i,j}} \times \mathbf{y}_{train(i)}\right) \tag{16}$$

where *n* is the number of training samples, and $\mathbf{y}_{train(i)}$ represents the true value of the *i*-th training sample.

**Algorithm 2.** The pseudo of S-*k*NN algorithm.

```
Input: X, Y;
Output:
switch task do
    case 1
        Class labels;
    endsw
    case 2
        Predicted value;
    endsw
    case 3
        Imputation value;
    endsw
endsw
Normalizing X and Y (When Y is class labels without normalization);
Optimizing Eq. (7) to obtain the optimal solution W;
Obtaining the optimal k value for test samples based on W;
switch task do
    case 1
        Obtaining class labels via majority rule;
    endsw
    case 2
        Obtaining prediction value via Eq. (16);
    endsw
    case 3
        Obtaining imputation value via Eq. (16);
    endsw
endsw
```

On the other hand, in the classification task, the proposed S-$k$NN algorithm uses the majority rule of *k* nearest neighbors of each test sample to predict the class label of each test sample.

## 4 Experimental Analysis

We evaluated the proposed S-$k$NN method in the tasks, such as classification, regression and missing value imputation, by comparing with the state-of-the-art $k$NN algorithms. Note that the classification tasks included binary classification and multi-class classification.

### 4.1 Experimental Setting

In our experiments, we regarded the standard $k$NN algorithm (with $k = 5$) as the first comparison algorithm. The second comparison algorithm is Eq. (7) with the setting $\rho_1 = 0$, *i.e.,* via LASSO learning different *k* values for test samples. We called this

algorithm LASSO based $k$NN algorithm, L-$k$NN for short, with which we would like to show the importance of preserving the local structures of data.

The used datasets came from UCI [1], LIBSVM [3] and the website[2]. We conducted experiments on four datasets for classification, regression, and missing value imputation, respectively. We coded all algorithms with MATLAB 7.1 in windows 7 system. We conducted experiments by10-fold cross-validation method. In this way, we repeated the whole process 10 times to avoid the possible bias in our experiments.

We used classification accuracy as the evaluation for the classification task. The higher accuracy the algorithm is, the better the performance of the classification is.

We employed Root Mean Square Error (RMSE) [31] and correlation coefficient to evaluate the performance of both regression analysis and missing value imputation. Note that there are not missing values in the original datasets, we randomly selected some independent values to be missed according to the literatures on missing value imputation [27].

RMSE is defined as the square root of predicted value and the ground-truth. Its formula is as follows:

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{\mathrm{i}=1}^{n}(\mathbf{y}_i - \hat{\mathbf{y}}_i)^2} \tag{17}$$

where $\mathbf{y}_i$ indicates the ground-truth, $\hat{\mathbf{y}}_i$ indicates the predicted value. Obviously, the smaller the RMSE is, the better the algorithm is.

Correlation coefficient indicates the correlation between prediction and observation. The range of correlation coefficient is between +1 and -1 inclusive, where 1 is total positive correlation, 0 is no correlation, and -1 is total negative correlation. Generally, the larger the correlation coefficient is, the more accurate the prediction is.

### 4.2 Results

In this section, we compared the performance of the proposed S-$k$NN algorithm to two comparison algorithms on real datasets, in terms of three tasks, such as classification, regression, and missing value imputation.

**Classification Results.** We summarized the classification accuracy results of all algorithms in Table 1. As shown in Table 1, we had the following observations:

- The proposed S-$k$NN algorithm outperformed the comparison algorithms, such as L-$k$NN and $k$NN. More specifically, the proposed S-$k$NN algorithm averagely improved by 5.97% and 16.32%, respectively, than the L-$k$NN algorithm and the $k$NN algorithm, in terms of classification accuracy.
- The S-$k$NN algorithm performed better than the L-$k$NN algorithm because the proposed S-$k$NN method used the LPP regularization term to preserve the local structures of data. In particular, the proposed method improved by 11.48% than the L-$k$NN, on Heart dataset. Both the S-$k$NN and the L-$k$NN outperformed the $k$NN algorithm. This indicates that using different k values in $k$NN algorithm (such as

[2] http://www.cc.gatech.edu/~lsong/code.html

**Table 1.** Comparison of classification accuracy

| Dataset | $k$NN | L-$k$NN | S-$k$NN |
|---|---|---|---|
| Cleveland | 0.7048 ± 0.0029 | 0.7619 ± 0.0030 | **0.8048 ± 0.0038** |
| Ionnosophere | 0.6743 ± 0.0031 | 0.8286 ± 0.0025 | **0.8571 ± 0.0025** |
| Heart | 0.5381 ± 0.0031 | 0.6741 ± 0.0033 | **0.7889 ± 0.0043** |
| Seeds | 0.8048 ± 0.0033 | 0.8714 ± 0.0015 | **0.9238 ± 0.0011** |

**Table 2.** Comparison of RMSE

| Dataset | $k$NN | L-$k$NN | S-$k$NN |
|---|---|---|---|
| Mpg | 3.8080 ± 0.4417 | 3.5909 ± 0.3662 | **3.4693 ± 0.3158** |
| Triazines | 0.1477 ± 0.0017 | 0.1357 ± 0.0016 | **0.1242 ± 0.0013** |
| Concreteslup | 0.0176 ± 0.000036 | 0.0151 ± 0.000033 | **0.0139 ± 0.0000339** |
| Bodyfat | 0.000021± 3.7921e-09 | 0.000014 ± 3.3355e-09 | **0.000013 ± 3.8892e-09** |

the S-$k$NN algorithm and the L-$k$NN algorithm) can achieve better classification performance than the method with fixed k value for all test samples, such as the conventional $k$NN algorithm.

**Regression Results.** We summarized the results of RMSE and correlation coefficient in Tables 2 and 3, respectively.

From Tables 2 and 3, we found that our proposed S-$k$NN achieved the best performance in terms of both RMSE and correlation coefficient, followed by L-$k$NN and $k$NN. In the evaluation of RMSE, the proposed S-$k$NN averagely reduced by 0.0915 and 0.0336 than the L-$k$NN and the $k$NN, on four datasets. In particular, the proposed S-$k$NN algorithm made the most improvement on Mpg dataset, *i.e.,* reduced by 0.1216 and 0.3387, than the L-$k$NN and the $k$NN. In terms of correlation coefficient, the proposed S-$k$NN averagely increased by 3.66% than L-$k$NN, and by10.42% than the $k$NN, on four datasets. Moreover, the proposed method achieved the maximal increase on Triazines dataset, *i.e.,* 8.8% compared to the L-$k$NN and 22.85% compared to the $k$NN.

Based on the results on regression analysis in our experiments, we can know: On one hand, it is reasonable for the proposed method to take the local structures of data into account in regression analysis. On the other hand, using different *k* values in $k$NN algorithm is practical. Moreover, the *k* values should be learnt from the data.

**Table 3.** Comparison of correlation coefficient

| Dataset | $k$NN | L-$k$NN | S-$k$NN |
|---|---|---|---|
| Mpg | 0.8865 ± 0.0019 | 0.8978 ± 0.0014 | **0.9076 ± 0.0011** |
| Triazines | 0.4256 ± 0.0148 | 0.5661 ± 0.0123 | **0.6541 ± 0.0106** |
| Concreteslup | 0.6606 ± 0.0312 | 0.7719 ± 0.0249 | **0.8194 ± 0.0195** |
| Bodyfat | 0.9846 ± 0.000081 | 0.9918 ± 0.000044 | **0.9930 ± 0.000043** |

**Table 4.** Comparison of RMSE

| Dataset | $k$NN | L-$k$NN | S-$k$NN |
|---|---|---|---|
| Abalone | 2.3894 ± 0.1019 | 2.1261± 0.1360 | **2.0850 ± 0.1207** |
| Housing | 5.2228 ± 1.2315 | 3.8666 ± 0.2108 | **3.7948 ± 0.2175** |
| Pyrim | 0.0673 ± 0.0002 | 0.0492 ± 0.0003 | **0.0484 ± 0.0003** |
| YachtHydrod | 10.5379 ± 3.0492 | 10.1437 ± 2.6463 | **9.4637 ± 3.2310** |

**Table 5.** Comparison of correlation coefficient

| Dataset | $k$NN | L-$k$NN | S-$k$NN |
|---|---|---|---|
| Abalone | 0.6499± 0.0008 | 0.7282 ± 0.0024 | **0.7376 ± 0.0021** |
| Housing | 0.8285 ± 0.0004 | 0.9142 ± 0.0004 | **0.9197 ± 0.0003** |
| Pyrim | 0.8283 ± 0.0079 | 0.9140 ± 0.0025 | **0.9201 ± 0.0022** |
| YachtHydrod | 0.7200 ± 0.0044 | 0.7948 ± 0.0039 | **0.8162 ± 0.0034** |

**Imputation Results.** We summarized the RMSE and the correlation coefficient, respectively, of the results of missing value imputation on all algorithms in Tables 4 and 5.

From Tables 4 and 5, we found that our proposed S-$k$NN achieved the best performance in terms of RMSE and correlation coefficient, followed by L-$k$NN and $k$NN. Regarding the case that the proposed S-$k$NN outperformed L-$k$NN, we knew that the preservation of local structures of data is necessary on missing value imputation with $k$NN algorithm. By comparing the S-$k$NN and the L-$k$NN with the $k$NN, we found that learning the optimal *k* value in $k$NN algorithm is feasible and useful.

In a word, according to the results on three learning tasks, we can make the following conclusion: it might be reasonable to use varied *k* values in $k$NN algorithm in real applications, while the optimal *k* values should be learnt from the data, *i.e.,* the data-driven *k* value in $k$NN algorithm.

## 5 Conclusion

In this work, we have proposed a new $k$NN algorithm by replacing the fixed *k* value for all test samples by learning the optimal *k* value for each test samples according to the distribution of data. To achieve this, we reconstructed test samples by training samples to obtain sparse correlation between each test sample and training samples. Moreover, the sparse correlation decided the *k* value for each test sample. Furthermore, we used the new $k$NN algorithm to conduct three learning tasks, such as classification, regression, and missing value imputation. The experimental results on real datasets verified the benefit of the proposed method, by comparing with the state-of-the-art $k$NN algorithms.

**Acknowledgements.** This work was supported in part by the National Natural Science Foundation of China under grants 61170131, 61263035 and 61363009, the China 863 Program under grant 2012AA011005, the China 973 Program under grant

2013CB329404, the Guangxi Natural Science Foundation under grant 2012GXNSFGA060004, the Key Project for Guangxi Universities' Science and Technology Research under grant 2013ZD041.

## References

1. Bache, K., Lichman, M.: UCI machine learning repository (2013)
2. Burba, F., Ferraty, F., Vieu, P.: k-nearest neighbour method in functional nonparametric regression. Journal of Nonparametric Statistics 21(4), 453–469 (2009)
3. Chang, C.-C., Lin, C.-J.: LIBSVM: A library for support vector machines. ACM Transactions on Intelligent Systems and Technology 2, 27:1–27:27 (2011), `http://www.csie.ntu.edu.tw/~cjlin/libsvm`
4. Cover, T., Hart, P.: Nearest neighbor pattern classification. IEEE Transactions on Information Theory 13(1), 21–27 (1967)
5. Ferraty, F., Vieu, P.: Nonparametric functional data analysis: theory and practice (2006)
6. Goldberger, J., Roweis, S.T., Hinton, G.E., Salakhutdinov, R.: Neighbourhood components analysis. In: NIPS (2004)
7. He, X., Niyogi, P.: Locality preserving projections. In: NIPS (2003)
8. Kang, P., Cho, S.: Locally linear reconstruction for instance-based learning. Pattern Recognition 41(11), 3507–3518 (2008)
9. Lall, U., Sharma, A.: A nearest neighbor bootstrap for resampling hydrologic time series. Water Resources Research 32(3), 679–693 (1996)
10. Liu, H., Zhang, S., Zhao, J., Zhao, X., Mo, Y.: A new classification algorithm using mutual nearest neighbors. In: GCC, pp. 52–57 (2010)
11. Meesad, P., Hengpraprohm, K.: Combination of knn-based feature selection and knnbased missing-value imputation of microarray data. In: ICICIC, pp. 341–341 (2008)
12. Qin, Y., Zhang, S., Zhu, X., Zhang, J., Zhang, C.: Semi-parametric optimization for missing data imputation. Applied Intelligence 27(1), 79–88 (2007)
13. Qin, Z., Wang, A.T., Zhang, C., Zhang, S.: Cost-sensitive classification with k-nearest neighbors. In: Wang, M. (ed.) KSEM 2013. LNCS, vol. 8041, pp. 112–131. Springer, Heidelberg (2013)
14. Tibshirani, R.: Regression shrinkage and selection via the lasso. Journal of the Royal Statistical Society. Series B (Methodological), 267–288 (1996)
15. Weinberger, K.Q., Blitzer, J., Saul, L.K.: Distance metric learning for large margin nearest neighbor classification. In: NIPS, pp. 1473–1480 (2005)
16. Wu, X., Zhang, C., Zhang, S.: Efficient mining of both positive and negative association rules. ACM Transactions on Information Systems (TOIS) 22(3), 381–405 (2004)
17. Wu, X., Zhang, C., Zhang, S.: Database classification for multi-database mining. Information Systems 30(1), 71–88 (2005)
18. Wu, X., Zhang, S.: Synthesizing high-frequency rules from different data sources. IEEE Transactions on Knowledge and Data Engineering 15(2), 353–367 (2003)
19. Zhang, C., Zhu, X., Zhang, J., Qin, Y., Zhang, S.: GBKII: An imputation method for missing values. In: Zhou, Z.-H., Li, H., Yang, Q. (eds.) PAKDD 2007. LNCS (LNAI), vol. 4426, pp. 1080–1087. Springer, Heidelberg (2007)
20. Zhang, S.: Cost-sensitive classification with respect to waiting cost. Knowledge-Based Systems 23(5), 369–378 (2010)
21. Zhang, S.: Estimating semi-parametric missing values with iterative imputation. International Journal of Data Warehousing and Mining 6(3), 1–10 (2010)

22. Zhang, S.: KNN-CF approach: Incorporating certainty factor to knn classification. IEEE Intelligent Informatics Bulletin 11(1), 24–33 (2010)
23. Zhang, S.: Shell-neighbor method and its application in missing data imputation. Applied Intelligence 35(1), 123–133 (2011)
24. Zhang, S.: Decision tree classifiers sensitive to heterogeneous costs. Journal of Systems and Software 85(4), 771–779 (2012)
25. Zhang, S.: Nearest neighbor selection for iteratively knn imputation. Journal of Systems and Software 85(11), 2541–2552 (2012)
26. Zhang, S., Jin, Z., Zhu, X.: Missing data imputation by utilizing information within incomplete instances. Journal of Systems and Software 84(3), 452–459 (2011)
27. Zhang, S., Jin, Z., Zhu, X., Zhang, J.: Missing data analysis: A kernel-based multi-imputation approach. In: Gavrilova, M.L., Tan, C.J.K. (eds.) Transactions on Computational Science III. LNCS, vol. 5300, pp. 122–142. Springer, Heidelberg (2009)
28. Zhang, S., Qin, Z., Ling, C.X., Sheng, S.: "Missing is useful": missing values in cost-sensitive decision trees. IEEE Transactions on Knowledge and Data Engineering 17(12), 1689–1693 (2005)
29. Zhao, Y., Zhang, S.: Generalized dimension-reduction framework for recent-biased time series analysis. IEEE Transactions on Knowledge and Data Engineering 18(2), 231–244 (2006)
30. Zhu, X., Huang, Z., Cheng, H., Cui, J., Shen, H.T.: Sparse hashing for fast multimedia search. ACM Transactions on Information Systems 31(2), 9 (2013)
31. Zhu, X., Huang, Z., Cui, J., Shen, H.T.: Video-to-shot tag propagation by graph sparse group lasso. IEEE Transactions on Multimedia 15(3), 633–646 (2013)
32. Zhu, X., Huang, Z., Shen, H.T., Zhao, X.: Linear cross-modal hashing for efficient multimedia search. In: ACM Multimedia, pp. 143–152 (2013)
33. Zhu, X., Huang, Z., Tao Shen, H., Cheng, J., Xu, C.: Dimensionality reduction by mixed kernel canonical correlation analysis. Pattern Recognition 45(8), 3003–3016 (2012)
34. Zhu, X., Huang, Z., Yang, Y., Tao Shen, H., Xu, C., Luo, J.: Self-taught dimensionality reduction on the high-dimensional small-sized data. Pattern Recognition 46(1), 215–229 (2013)
35. Zhu, X., Suk, H.-I., Shen, D.: Matrix-similarity based loss function and feature selection for alzheimer's disease diagnosis. In: CVPR, pp. 3089–3096 (2014)
36. Zhu, X., Suk, H.-I., Shen, D.: A novel matrix-similarity based loss function for joint regression and classification in ad diagnosis. NeuroImage (2014)
37. Zhu, X., Zhang, L., Huang, Z.: A sparse embedding and least variance encoding approach to hashing. IEEE Transactions on Image Processing 23(9), 3737–3750 (2014)
38. Zhu, X., Zhang, S., Jin, Z., Zhang, Z., Xu, Z.: Missing value estimation for mixed-attribute data sets. IEEE Transactions on Knowledge and Data Engineering 23(1), 110–121 (2011)